

ONEDRAFT

CAD-AI — Aria Powered

Executable Application Stack & Implementation Architecture

Version 1.0

Status: Implementation Specification

Predecessor: OneDraft Database Schema & OpenAPI Contract v0.1

Database: PostgreSQL

API: REST / JSON

API Version: /api/v1

Identifiers: UUID

Primary Model: Versioned OneDraft Project Model

AI Layer: Aria Powered

Execution Model: API + Domain Services + Workers + Geometry + Compliance + Documentation

Implementation State: Executable Foundation Architecture

1. IMPLEMENTATION OBJECTIVE

The OneDraft persistence and API contract established the system's machine interface and transactional memory.

The next layer converts that contract into an executable software system.

The implementation stack shall therefore provide the runtime required to execute:

```
CLIENT
  ↓
WEB APPLICATION
  ↓
API
  ↓
AUTHORIZATION
  ↓
DOMAIN SERVICES
  ↓
PROJECT MODEL
  ↓
POSTGRESQL
  ↓
WORK QUEUE
  ↓
WORKERS
  ↓
GEOMETRY
```

↓
VALIDATION
↓
DOCUMENTATION
↓
OBJECT STORAGE

Aria operates through the same controlled application boundary.

Aria does not become the database.

Aria does not become the project model.

Aria operates the project model through defined domain capabilities.

2. APPLICATION STACK

The initial OneDraft implementation shall consist of:

```
apps/  
  web/  
  api/  
  worker/  
  
packages/  
  domain/  
  database/  
  geometry/  
  compliance/  
  documentation/  
  ai/  
  storage/  
  events/  
  shared/  
  
docs/  
  openapi/  
  architecture/  
  specifications/  
  
migrations/  
  
tests/
```

The repository therefore separates:

user interface

API transport

background execution

domain logic

persistence

geometry

regulatory verification

documentation

AI orchestration

storage

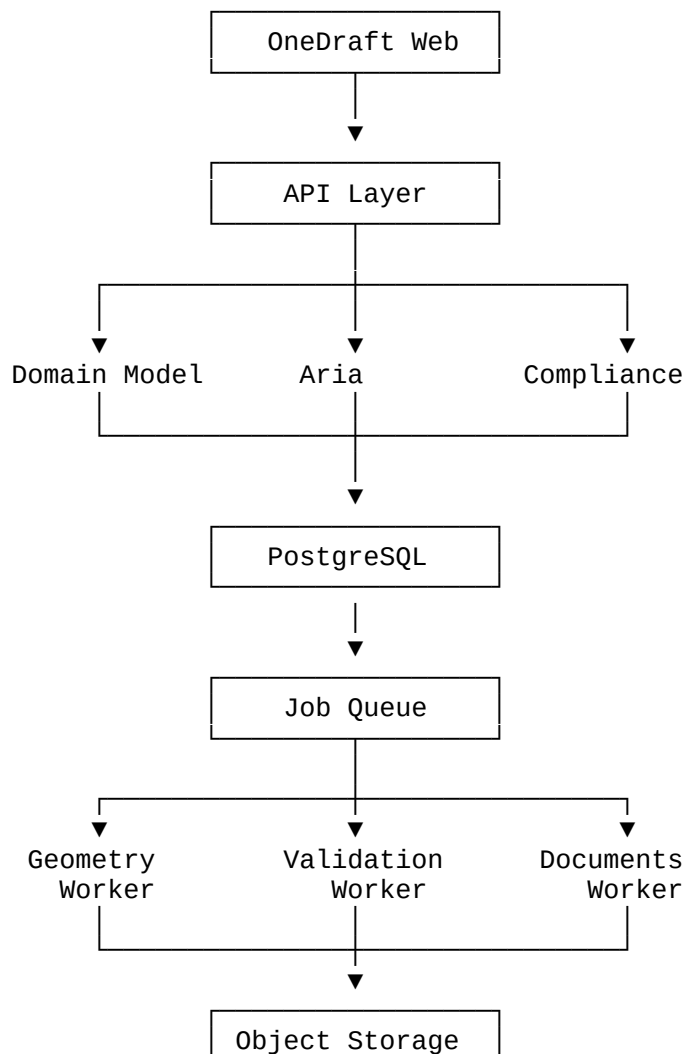
events

and:

shared contracts

3. RUNTIME ARCHITECTURE

The production runtime is:



4. WEB APPLICATION

The web application is the primary interactive client.

Initial responsibilities:

- authentication
- organization selection
- project selection
- project creation
- project dashboard
- model inspection
- drawing inspection
- redline creation
- Aria interaction
- validation results
- document package review
- acceptance

The web application must not implement domain rules independently of the API.

The API remains authoritative.

5. WEB APPLICATION STRUCTURE

Initial structure:

apps/web/

- src/
 - app/
 - components/
 - features/
 - projects/
 - model/
 - drawings/
 - redlines/
 - validation/
 - documents/
 - aria/
 - api/
 - hooks/
 - state/
 - types/
 - utils/

public/

package.json

tsconfig.json

The frontend shall consume the generated TypeScript API client.

6. FRONTEND DOMAIN BOUNDARY

The frontend may maintain:

- UI state
- selection state
- viewport state
- temporary interaction state
- form state
- optimistic state

The frontend shall not become the authoritative source for:

- project state
- revision state
- code compliance
- model geometry
- validation results
- document provenance
- acceptance state

Those remain server-side.

7. API APPLICATION

The API application provides the external machine boundary.

Initial structure:

```
apps/api/  
  app/  
    main.py  
  api/  
    routes/  
    dependencies/  
    middleware/  
  services/  
  security/  
  schemas/  
  settings/
```

tests/

pyproject.toml

The API layer is responsible for:

HTTP
authentication
authorization
request validation
response serialization
idempotency
transaction coordination
job submission
error handling
observability

8. API FRAMEWORK

The initial API implementation shall use a Python ASGI architecture.

The implementation shall support:

REST
JSON
OpenAPI
async request handling
dependency injection
request validation
typed responses

The generated OpenAPI document remains the public contract.

9. API STARTUP

The application entry point shall conceptually be:

apps/api/app/main.py

Responsibilities:

create application
load configuration
initialize logging
initialize database
initialize service registry
register middleware
register routes
register exception handlers
expose health endpoints

10. HEALTH ENDPOINTS

The API shall expose:

GET /health
GET /ready

Health determines whether the process is alive.

Readiness determines whether required dependencies are available.

The readiness check may verify:

PostgreSQL
queue
object storage
required internal services

AI providers shall not necessarily be required for basic API readiness.

11. CONFIGURATION

Configuration shall be environment-driven.

Initial environment categories:

APPLICATION
DATABASE
AUTHENTICATION
QUEUE
OBJECT_STORAGE
AI
COMPLIANCE
DOCUMENTATION
OBSERVABILITY

Secrets must never be committed to source control.

The repository shall contain:

.env.example

but never:

.env

with production secrets.

12. DATABASE ACCESS

The domain layer shall never construct raw database connections.

Database access is provided through the persistence package.

Conceptually:

```
Domain Service
  ↓
Repository / Unit of Work
  ↓
Database Session
  ↓
PostgreSQL
```

This creates a controlled persistence boundary.

13. DATABASE PACKAGE

Initial structure:

```
packages/database/
```

```
    migrations/
    models/
    repositories/
    session/
    unit_of_work/
    queries/
```

The package provides:

```
database session
transaction management
ORM mappings
repositories
query services
migration integration
```

14. UNIT OF WORK

Every command that modifies the project model shall execute through a transaction boundary.

Conceptually:

```
with unit_of_work() as uow:
```

```
    project = uow.projects.get(project_id)
```



```
command = validate_command(...)

result = project_model.execute(command)

uow.commit()
```

If execution fails:

ROLLBACK

No partial state is permitted.

15. DOMAIN PACKAGE

The domain package is the core of OneDraft.

Initial structure:

```
packages/domain/

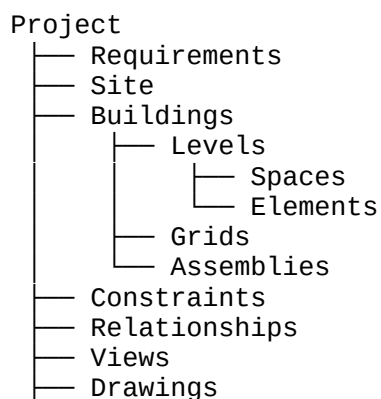
    entities/
    value_objects/
    commands/
    events/
    services/
    repositories/
    policies/
    errors/
    state/
```

The domain package contains the semantic rules of the OneDraft project model.

16. PROJECT MODEL

The Project Model is the authoritative computational representation of the project.

Conceptually:



The Project Model is versioned.

17. DOMAIN ENTITY RULE

Every domain entity must have:

UUID
identity
lifecycle state
validation rules
serialization boundary

Mutable entities must also support:

created_at
updated_at

Where applicable:

created_by
updated_by

18. VALUE OBJECTS

Repeated semantic concepts shall be represented as value objects.

Initial candidates:

Length
Area
Volume
Angle
Elevation
Coordinate
BoundingBox
Unit
Scale
ProjectAddress
Jurisdiction
RevisionNumber
DrawingNumber
SheetNumber

This prevents unit and semantic ambiguity from propagating through the application.

19. UNIT SYSTEM

OneDraft shall support:

METRIC
IMPERIAL

The internal computational representation should use a consistent canonical unit system.

User-facing values may be converted at the presentation boundary.

The conversion layer must preserve precision.

20. COMMAND MODEL

All model mutations are represented as commands.

Initial commands include:

CreateProject
CreateRequirement
CreateBuilding
CreateLevel
CreateSpace
CreateElement
UpdateElement
DeleteElement
MoveElement
CreateConstraint
UpdateConstraint
CreateRelationship
GenerateView
GenerateDrawing
CreateRedline
ResolveRedline
RunValidation
GenerateDocumentPackage
AcceptRevision

Commands are explicit application operations.

21. COMMAND HANDLER

Each command shall have a defined handler.

Conceptually:

Command
↓
Authorization

↓
Load Aggregate
↓
Validate Preconditions
↓
Apply Mutation
↓
Record Change
↓
Create Revision
↓
Emit Events
↓
Commit

22. COMMAND PRECONDITIONS

Commands must verify:

project exists
user authorized
revision is current
target objects exist
target objects are compatible
constraints permit operation
command parameters are valid

The system must reject stale mutations.

23. OPTIMISTIC CONCURRENCY

Commands that mutate a project must include an expected revision.

Example:

`expected_revision = 14`

If the current project revision is:

15

the command fails with:

`STALE_REVISION`

The client must retrieve the current state before retrying.

24. DOMAIN EVENTS

Successful domain operations produce events.

Initial event types:

ProjectCreated
RequirementCreated
BuildingCreated
LevelCreated
SpaceCreated
ElementCreated
ElementModified
ElementDeleted
ConstraintCreated
CommandExecuted
RevisionCreated
RedlineCreated
RedlineResolved
ValidationCompleted
DrawingGenerated
DocumentPackageGenerated
AcceptanceRecorded

Events are persisted through the audit/event subsystem.

25. EVENT PAYLOAD

An event shall contain:

event_id
event_type
project_id
aggregate_id
revision_id
actor_id
timestamp
payload

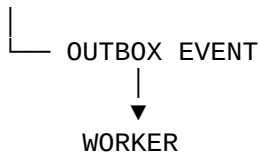
Events must be immutable.

26. EVENT OUTBOX

The application shall use an outbox pattern for reliable asynchronous processing.

Conceptually:

DATABASE TRANSACTION
|
— MODEL CHANGE



The event and model mutation commit atomically.

A worker then processes the event.

27. JOB SYSTEM

Expensive operations are asynchronous.

Initial job types:

GEOMETRY_GENERATION
VALIDATION
DRAWING_GENERATION
DOCUMENT_GENERATION
EXPORT
CODE_INGESTION

Every job receives:

job_id
project_id
revision_id
model_version_id
job_type
status
created_at
started_at
completed_at

28. WORKER APPLICATION

Initial structure:

apps/worker/

worker.py

jobs/

geometry.py
validation.py
drawings.py
documents.py
compliance.py

handlers/

```
services/  
tests/
```

Workers are stateless execution processes.

They retrieve authoritative state from the application/database and produce derived outputs.

29. WORKER SAFETY

Workers must verify:

```
project_id  
revision_id  
model_version_id  
job state  
input hashes
```

before producing an output.

A worker must never silently overwrite a newer model state.

30. STALE JOB PROTECTION

If:

```
job.model_version_id != current.model_version_id
```

the worker shall mark the result:

STALE

or:

INVALIDATED

depending on the workflow.

The stale output cannot become current project state.

31. GEOMETRY ENGINE

The geometry subsystem converts semantic model entities into computational geometry.

Its responsibilities include:

```
2D geometry
```

3D geometry
topology
intersections
offsets
boolean operations
spatial relationships
bounding volumes
geometry validation
geometry versioning

The geometry engine is independent of the web UI.

32. GEOMETRY REPRESENTATION

The architecture shall distinguish:

semantic element

from:

computed geometry

For example:

wall

is the semantic object.

Its generated geometry may include:

centerline
profile
solid
faces
edges
openings
connections

33. GEOMETRY PIPELINE

The initial pipeline is:

```
MODEL ELEMENT
  ↓
PARAMETERS
  ↓
GEOMETRY SOLVER
  ↓
TOPOLOGY
  ↓
```


GEOMETRY VALIDATION



GEOMETRY VERSION



OBJECT STORAGE / GEOMETRY STORE

34. GEOMETRY HASHING

Every computed geometry payload receives a deterministic hash.

Conceptually:

```
geometry_hash =  
HASH(  
    model_version_id  
    +  
    element_id  
    +  
    geometry_parameters  
)
```

This permits:

- deduplication
- cache validation
- reproducibility
- stale-output detection

35. GEOMETRY CACHE

Repeated geometry calculations should be cacheable.

The cache key shall incorporate:

- element_id
- model_version_id
- geometry_version
- parameter_hash

The cache must never override authoritative model state.

36. SPATIAL INDEX

The geometry subsystem should maintain a spatial acceleration structure for:

- collision detection
- intersection queries

selection
view generation
section generation
spatial validation

The initial implementation may use bounding volumes before introducing more advanced spatial indexing.

37. MODEL DERIVATION

The Project Model remains semantic.

Derived geometry is generated from:

element type
parameters
assemblies
relationships
constraints
level elevation
coordinate system

Therefore:

MODEL
↓
GEOMETRY

and not:

GEOMETRY
↓
MODEL

38. ARIA ORCHESTRATION

Aria is implemented as an orchestration layer.

Initial structure:

packages/ai/
 orchestration/
 prompts/
 adapters/
 tools/
 policies/
 schemas/
 memory/

Aria receives project context and user intent.

It returns structured operations.

39. ARIA EXECUTION MODEL

The workflow is:

```
graph TD
    A[USER REQUEST] --> B[ARIA]
    B --> C[INTENT ANALYSIS]
    C --> D[PROJECT CONTEXT]
    D --> E[CONSTRAINT ANALYSIS]
    E --> F[COMMAND PROPOSAL]
    F --> G[DOMAIN VALIDATION]
    G --> H[EXECUTION]
    H --> I[VALIDATION]
    I --> J[RESULT]
```

Aria does not directly mutate the database.

40. ARIA TOOL BOUNDARY

Aria receives defined tools such as:

```
get_project
get_revision
get_element
query_relationships
query_constraints
propose_command
run_validation
generate_drawing
create_redline
resolve_redline
```

Tools return structured application data.

41. ARIA COMMAND APPROVAL

Commands may operate in:

PROPOSE
EXECUTE

modes.

High-impact operations may require explicit approval.

Examples:

major structural modification
large-scale geometry change
code-sensitive modification
final document generation
customer acceptance

The approval policy remains application-controlled.

42. ARIA CONFIDENCE

Aria may return:

confidence
reasoning_summary
affected_objects
anticipated_impacts
required_validation

The confidence value is advisory.

Domain validation remains authoritative.

43. ARIA PROVIDER ADAPTER

The underlying AI model shall be hidden behind:

AIProviderAdapter

Conceptually:

Aria
↓
Provider Adapter
↓
Model Provider

This permits the underlying model provider to change without changing OneDraft's public API.

44. AI CONTEXT BUILDER

Aria shall not receive the entire database indiscriminately.

The context builder selects relevant:

```
project data
current revision
affected objects
constraints
requirements
code information
drawing context
redlines
prior command results
```

Context is assembled according to the task.

45. AI OUTPUT SCHEMA

Aria's structured output shall conform to a machine-readable schema.

Conceptually:

```
{
  "operation": "MOVE_ELEMENT",
  "targets": [],
  "parameters": {},
  "preserve_constraints": [],
  "validation_requirements": [],
  "confidence": 0.0
}
```

Unstructured AI output cannot directly become a model mutation.

46. DOMAIN VALIDATION OF AI OUTPUT

The command pipeline is:

```
AI OUTPUT
↓
SCHEMA VALIDATION
↓
AUTHORIZATION
↓
```

DOMAIN VALIDATION



CONSTRAINT VALIDATION



EXECUTION

A valid AI response is not automatically a valid architectural operation.

47. COMPLIANCE ENGINE

The compliance subsystem evaluates project state against the applicable code manifest.

Initial structure:

packages/compliance/

```
sources/  
ingestion/  
rules/  
applicability/  
evaluation/  
manifests/  
citations/
```

48. CODE INGESTION

Code sources enter the system through a controlled ingestion process.

The process is:

SOURCE



RETRIEVAL



IDENTIFICATION



VERSION



HASH



NORMALIZATION



RULE EXTRACTION



REVIEW



CODE SOURCE

49. CODE PROVENANCE

Every regulatory rule must retain provenance.

Minimum provenance:

jurisdiction
authority
source
version
effective_date
retrieved_at
content_hash
citation

A validation result must be traceable to the applicable rule.

50. CODE MANIFEST GENERATION

For every project:

PROJECT
↓
JURISDICTION
↓
APPLICABLE SOURCES
↓
CODE MANIFEST
↓
APPLICABLE RULES

The manifest is attached to the relevant revision.

51. COMPLIANCE EVALUATION

The compliance engine evaluates:

project requirements
occupancy
building type
construction type
area
height
levels
egress
accessibility
spatial requirements
fire protection
structural interfaces
documentation requirements

The exact rule categories expand according to jurisdiction.

52. COMPLIANCE RESULT

Each evaluation produces:

PASS
FAIL
WARNING
NOT_APPLICABLE
UNDETERMINED

Every result contains:

rule
affected objects
description
severity
source

53. VALIDATION ENGINE

Validation is broader than regulatory compliance.

The validation system includes:

CODE
SPATIAL
GEOMETRY
COORDINATION
ACCESSIBILITY
DOCUMENT
MODEL

The engine aggregates results into the revision's validation state.

54. VALIDATION GATE

A final document package shall not be marked final unless required validation gates have passed.

Conceptually:

MODEL
↓
VALIDATION
↓
NO BLOCKING FAILURES

↓
DRAWING GENERATION
↓
DOCUMENT VALIDATION
↓
FINAL PACKAGE

Blocking and non-blocking findings are distinguished.

55. DRAWING ENGINE

The drawing subsystem converts the computational model into architectural documentation.

Responsibilities:

view generation
projection
linework
annotation
dimensioning
symbol placement
schedules
sheet composition
title blocks
revision blocks
graphic standards

56. VIEW GENERATION

Initial view types:

PLAN
ELEVATION
SECTION
DETAIL
3D
SCHEDULE
DIAGRAM

A view is always associated with a model version.

57. SECTION GENERATION

A section consists of:

section plane
cut geometry

projection geometry
visibility rules
line hierarchy
annotations
dimensions
references

The section must be regenerated when the underlying model state changes.

58. PLAN GENERATION

A plan view is derived from:

level
cut plane
model geometry
visibility settings
annotation rules
dimension rules

The plan is not manually redrawn as a separate source of truth.

59. ELEVATION GENERATION

Elevations are derived from:

building geometry
orientation
projection rules
level information
opening geometry
annotation standards

The elevation remains linked to the underlying model.

60. ARCH D SHEETS

OneDraft shall support architectural sheet composition for:

ARCH D

The sheet engine manages:

paper size
title block
drawing placement
viewports

notes
schedules
revision block
sheet numbering

Additional sheet sizes may be introduced later.

61. DOCUMENT ENGINE

The document engine assembles derived documentation into deliverable packages.

Pipeline:

```
PROJECT
↓
REVISION
↓
VALIDATED MODEL
↓
DRAWINGS
↓
SHEETS
↓
DOCUMENT PACKAGE
↓
HASH
↓
OBJECT STORAGE
```

62. DOCUMENT PACKAGE CONTENT

A package may contain:

plans
elevations
sections
details
schedules
notes
cover sheet
code information
revision information
validation summary

The package manifest identifies every constituent artifact.

63. OBJECT STORAGE

Large generated artifacts shall not be stored directly inside PostgreSQL.

Object storage shall contain:

- PDF
- SVG
- DXF
- IFC
- images
- geometry payloads
- source artifacts
- generated packages

PostgreSQL stores references and metadata.

64. OBJECT STORAGE KEY

Objects should use deterministic project-aware paths.

Conceptually:

```
projects/{project_id}/  
  revisions/{revision_id}/  
    model/  
    geometry/  
    drawings/  
    documents/  
    validation/
```

65. DOCUMENT HASHING

Final document packages receive a cryptographic hash.

Conceptually:

```
document_hash =  
HASH(package_bytes)
```

The hash is stored with:

```
revision_id  
model_version_id  
code_manifest_id  
validation_run_id
```

66. DOCUMENT REPRODUCIBILITY

A package must be reproducible from its source state.

The reconstruction chain is:

```
PROJECT
↓
MODEL VERSION
↓
REVISION
↓
CODE MANIFEST
↓
VALIDATION
↓
DRAWINGS
↓
PACKAGE
```

If a package cannot be traced through this chain, it is not a complete OneDraft deliverable.

67. REDLINE ENGINE

Redlines are treated as model-change requests.

Workflow:

```
OPEN
↓
ANALYZED
↓
PROPOSED
↓
IMPLEMENTED
↓
VALIDATED
↓
RESOLVED
```

A redline is not considered resolved merely because an AI response has been generated.

68. REDLINE IMPLEMENTATION

A redline produces:

```
target object
command
revision
geometry changes
```

drawing changes
validation results

This creates traceability from user markup to final documentation.

69. REVISION ENGINE

Every accepted model mutation creates a revision according to the project's revision policy.

The revision records:

parent revision
model version
changes
actor
reason
code manifest
validation state

70. REVISION GRAPH

The model history is represented as:

Revision 1
↓
Revision 2
↓
Revision 3
↓
Revision 4

Future branching may support:

Revision 2
├─ Revision 3A
└─ Revision 3B

The initial implementation may remain linear.

71. PROJECT SNAPSHOT

A project snapshot is a consistent representation of:

project
requirements
model
constraints

relationships
current revision
current model version

Snapshots provide the input state for workers and AI context construction.

72. SERIALIZATION

Domain objects shall have explicit serialization boundaries.

The system must distinguish:

database representation
domain representation
API representation
AI representation
geometry representation
document representation

These representations must not be conflated.

73. ERROR MODEL

The application shall use typed domain errors.

Initial categories:

NOT_FOUND
UNAUTHORIZED
FORBIDDEN
INVALID_COMMAND
STALE_REVISION
CONSTRAINT_CONFLICT
VALIDATION_FAILED
GEOMETRY_FAILED
JOB_FAILED
DOCUMENT_GENERATION_FAILED
DEPENDENCY_UNAVAILABLE

Errors are mapped to standardized API responses.

74. OBSERVABILITY

The runtime shall provide:

structured logging
request IDs

job IDs
project IDs
revision IDs
model version IDs
metrics
health checks
error tracking

Every major operation should be traceable through its request/job identifier.

75. CORRELATION ID

Requests receive:

request_id

Background jobs receive:

job_id

Related events retain the originating identifiers.

This permits tracing:

```
USER REQUEST
↓
API REQUEST
↓
COMMAND
↓
REVISION
↓
JOB
↓
DRAWING
↓
DOCUMENT
```

76. SECURITY BOUNDARY

Security is enforced at:

API
DOMAIN
DATABASE
OBJECT STORAGE
WORKER

The system follows least privilege.

Services receive only the permissions required for their role.

77. DATABASE SERVICE ACCOUNT

The API should use a controlled database identity.

Workers should use appropriate restricted credentials.

The frontend receives no database credentials.

No external client receives PostgreSQL access.

78. OBJECT STORAGE SECURITY

Generated project documents must be protected by project authorization.

Object references must not be assumed to constitute authorization.

Downloads should be mediated through controlled access.

79. AUDIT SECURITY

Audit records are append-only.

Normal application users cannot modify historical event records.

Administrative capabilities must themselves produce audit events.

80. IDEMPOTENCY IMPLEMENTATION

Mutating API requests shall support:

Idempotency-Key

The API stores the request identity and resulting response.

A repeated request with the same key and same request hash returns the existing result.

A reused key with a different request hash is rejected.

81. API SERVICE LAYER

The API shall not contain complex domain logic inside route handlers.

Preferred structure:

```
Route
↓
Request Schema
↓
Application Service
↓
Domain Service
↓
Unit of Work
↓
Repository
↓
Database
```

This keeps HTTP concerns separate from domain behavior.

82. PROJECT MODEL SERVICE

The first major domain service shall be:

`ProjectModelService`

Initial responsibilities:

```
load project state
validate commands
apply commands
create revisions
record changes
emit events
return model results
```

This service becomes the principal command boundary for OneDraft.

83. INITIAL SERVICE INTERFACE

Conceptually:

```
execute_command(
    project_id,
    command,
    expected_revision,
    actor_id
)
```

The service returns:

```
command_id  
revision_id  
model_version_id  
affected_objects  
events
```

84. PROJECT CREATION SERVICE

The first project workflow is:

```
Create Organization  
↓  
Create User  
↓  
Create Membership  
↓  
Create Project  
↓  
Create Initial Model Version  
↓  
Create Initial Revision  
↓  
Emit ProjectCreated
```

The initial project must begin in a consistent state.

85. MODEL CREATION SERVICE

The first architectural model workflow is:

```
Project  
↓  
Building  
↓  
Level  
↓  
Space  
↓  
Elements  
↓  
Relationships  
↓  
Constraints
```

Each object is created through the domain boundary.

86. INITIAL ELEMENT TYPES

The first implementation shall support:

WALL
DOOR
WINDOW
FLOOR
ROOF
COLUMN
BEAM
STAIR
RAMP
SPACE
OPENING

Additional element classes are introduced incrementally.

87. ELEMENT PARAMETER MODEL

Elements may contain typed parameters such as:

width
height
thickness
length
offset
material
assembly
orientation
location
host

Parameters are validated according to element type.

88. WALL EXAMPLE

A wall may contain:

wall_type
length
height
thickness
base_level
top_level
location
orientation
assembly_id

The geometry engine converts these parameters into computational geometry.

89. DOOR EXAMPLE

A door may contain:

width
height
head_height
swing_direction
swing_angle
host_wall
fire_rating
accessibility_properties

The system derives door geometry and associated documentation.

90. WINDOW EXAMPLE

A window may contain:

width
height
sill_height
head_height
host_wall
window_type
performance_properties

The opening is coordinated with the host wall.

91. RELATIONSHIP ENGINE

Relationships become computational dependencies.

Examples:

DOOR → HOSTED_BY → WALL
WINDOW → HOSTED_BY → WALL
SPACE → BOUNDED_BY → WALL
ELEMENT → LOCATED_ON → LEVEL
DRAWING → REPRESENTS → MODEL_VERSION

When a parent changes, dependent objects can be identified.

92. DEPENDENCY PROPAGATION

A change to:

wall

may affect:

door
window
space boundary
plan
elevation
section
schedule
validation
document package

The dependency graph identifies affected derived outputs.

93. AFFECTED-OUTPUT CALCULATION

After a model mutation:

CHANGED OBJECT
↓
DEPENDENCY GRAPH
↓
AFFECTED OBJECTS
↓
AFFECTED VIEWS
↓
AFFECTED DRAWINGS
↓
AFFECTED DOCUMENTS

Only necessary derived artifacts should be regenerated where practical.

94. INCREMENTAL REGENERATION

The system shall prefer incremental regeneration.

Example:

wall changed

does not automatically require regeneration of unrelated:

site plan

unless the dependency graph determines that it is affected.

95. FULL REGENERATION

A full regeneration remains available.

Example:

```
POST /api/v1/projects/{project_id}/documents/generate
```

may request complete package regeneration.

Full regeneration is a recovery and verification mechanism.

96. BUILD SYSTEM

The repository shall support reproducible builds.

Initial requirements:

- dependency lock files
- container builds
- environment configuration
- database migrations
- test execution
- linting
- type checking

97. CONTAINERIZATION

The initial deployment shall support:

- web
- api
- worker
- postgres
- queue
- object-storage

through a development orchestration file.

Conceptually:

```
docker-compose.yml
```

Production deployment may later move to managed infrastructure.

98. DEVELOPMENT ENVIRONMENT

A developer should be able to execute:

```
clone repository
↓
install dependencies
↓
configure environment
↓
start services
↓
run migrations
↓
seed development database
↓
run tests
↓
start web application
```

without manually reconstructing infrastructure.

99. MIGRATION EXECUTION

Database initialization follows:

```
migration runner
↓
001_initial_schema
↓
subsequent migrations
↓
seed development data
```

Migrations are immutable once deployed.

100. TESTING PYRAMID

OneDraft testing shall contain:

```
unit tests
domain tests
repository tests
API tests
integration tests
worker tests
geometry tests
```


compliance tests
document tests
end-to-end tests

101. DOMAIN TESTS

Domain tests must verify:

valid commands
invalid commands
constraint conflicts
revision creation
dependency propagation
stale revisions
change records

These tests should not require the full web stack.

102. API TESTS

API tests verify:

authentication
authorization
request validation
response schema
idempotency
error handling
job creation
pagination

103. INTEGRATION TEST

The first major integration test remains:

```
CREATE ORGANIZATION
↓
CREATE USER
↓
CREATE PROJECT
↓
CREATE REQUIREMENTS
↓
CREATE BUILDING
↓
CREATE LEVELS
↓
```

CREATE ROOMS
↓
CREATE WALLS
↓
CREATE DOORS
↓
CREATE WINDOWS
↓
GENERATE DRAWINGS
↓
ARIA MODIFICATION
↓
VALIDATION
↓
REVISION
↓
REDLINE
↓
REDLINE RESOLUTION
↓
DOCUMENT PACKAGE
↓
HASH
↓
ACCEPTANCE
↓
HISTORY RECONSTRUCTION

104. RECONSTRUCTION TEST

The system must prove:

historical revision
=
reconstructed revision

using only persisted project state and associated immutable artifacts.

This becomes a fundamental persistence regression test.

105. DOCUMENT RECONSTRUCTION TEST

Given:

project_id
revision_id
model_version_id
code_manifest_id

the system must determine:

which model state existed
which code sources applied
which validation was executed
which drawings were generated
which package was produced

106. API CONTRACT TEST

The generated API client and server implementation must be tested against:

`openapi.yaml`

The contract becomes a build artifact rather than merely documentation.

107. GENERATED TYPES

The frontend TypeScript client shall be generated from OpenAPI.

The Python application models shall remain consistent with the same contract.

Where practical:

```
OpenAPI
  ↓
generated client types
  ↓
API implementation
```

The API contract remains the synchronization point.

108. DOMAIN TYPE SEPARATION

The domain model must not simply become the OpenAPI schema.

Instead:

Domain Types

represent business semantics.

API Schemas

represent transport semantics.

Adapters translate between them.

109. PACKAGE DEPENDENCY RULE

Dependency direction shall be controlled.

Preferred:

```
shared
  ↑
domain
  ↑
database / geometry / compliance / documentation / ai
  ↑
api / worker
  ↑
web
```

Lower-level packages must not depend on the web application.

110. DOMAIN INDEPENDENCE

The domain model should be executable without:

```
HTTP
AI provider
browser
PDF renderer
specific database driver
```

This is essential for deterministic testing.

111. AI INDEPENDENCE

The Project Model must remain fully functional if the AI provider is unavailable.

Aria accelerates interaction.

Aria is not the underlying system of record.

112. COMPLIANCE INDEPENDENCE

The model can exist without a completed compliance run.

However:

```
finalization
```

may require successful validation according to the project's configured workflow.

113. DOCUMENTATION INDEPENDENCE

Documentation is derived from model state.

A drawing must never become the only surviving representation of an architectural object.

114. STORAGE INDEPENDENCE

Object storage contains large artifacts.

PostgreSQL contains authoritative metadata and relationships.

The application coordinates the two.

115. IMPLEMENTATION REPOSITORY

The implementation repository shall contain at minimum:

```
OneDraft/
├── README.md
├── LICENSE
├── pyproject.toml
├── .env.example
├── docker-compose.yml
├── apps/
│   ├── web/
│   ├── api/
│   └── worker/
├── packages/
│   ├── domain/
│   ├── database/
│   ├── geometry/
│   ├── compliance/
│   ├── documentation/
│   ├── ai/
│   ├── storage/
│   ├── events/
│   └── shared/
├── migrations/
├── docs/
│   ├── openapi/
│   └── architecture/
```

└─ specifications/
└─ tests/

116. FIRST EXECUTABLE ARTIFACTS

The first implementation package shall contain:

migrations/001_initial_schema.sql

docs/openapi/openapi.yaml

packages/domain/types.py

packages/domain/commands.py

packages/domain/services/project_model_service.py

apps/api/app/main.py

apps/api/app/api/routes/projects.py

apps/worker/worker.py

117. FIRST DOMAIN IMPLEMENTATION

The first implemented command shall be:

CreateProject

It establishes the complete path:

HTTP

↓

API Schema

↓

Application Service

↓

Domain Command

↓

Unit of Work

↓

Repository

↓

PostgreSQL

↓

Revision

↓

Event

↓

API Response

118. SECOND DOMAIN IMPLEMENTATION

The second command shall be:

CreateBuilding

This establishes the relationship:

Organization

↓

Project

↓

Building

119. THIRD DOMAIN IMPLEMENTATION

The third command shall be:

CreateLevel

This establishes:

Building

↓

Level

120. FOURTH DOMAIN IMPLEMENTATION

The fourth command shall be:

CreateSpace

This establishes:

Level

↓

Space

121. FIFTH DOMAIN IMPLEMENTATION

The fifth domain operation shall establish architectural elements:

CreateElement

Initial supported elements:

WALL
DOOR
WINDOW

This creates the minimum viable architectural model.

122. FIRST GEOMETRY IMPLEMENTATION

The first geometry implementation shall support:

wall centerline
wall profile
wall solid
door opening
window opening
space boundary

The objective is not a complete CAD engine.

The objective is a deterministic architectural geometry foundation.

123. FIRST DRAWING IMPLEMENTATION

The first drawing generator shall produce:

floor plan

from the computational model.

The floor plan shall demonstrate:

walls
doors
windows
spaces
dimensions
annotations
level identification
sheet composition

124. FIRST SHEET IMPLEMENTATION

The first sheet shall be:

ARCH D

with:

title block
sheet number
drawing title
scale
revision
project information

The sheet is generated from model state.

125. FIRST ARIA IMPLEMENTATION

The first Aria operation shall be:

MoveElement

Example intent:

Move the rear bedroom wall 300 mm north.

Aria translates the request into:

MOVE_ELEMENT

with structured parameters.

126. FIRST VALIDATION IMPLEMENTATION

The first validation layer shall verify:

geometry validity
host relationships
space boundaries
required dimensions
model consistency

Regulatory validation can then be layered on top.

127. FIRST REDLINE IMPLEMENTATION

The first redline workflow shall support:

```
CREATE REDLINE
↓
TARGET OBJECT
↓
ARIA ANALYSIS
↓
COMMAND PROPOSAL
↓
COMMAND EXECUTION
↓
VALIDATION
↓
RESOLUTION
```

128. FIRST DOCUMENT IMPLEMENTATION

The first document package shall contain:

```
ARCH D floor plan
project information
revision information
validation summary
```

The package shall be hashed and persisted.

129. FIRST ACCEPTANCE IMPLEMENTATION

The acceptance workflow shall record:

```
customer
project
revision
model version
document package
code manifest
timestamp
acceptance status
acknowledgements
```

Acceptance does not mutate the historical model.

It records an event against a defined revision.

130. FINALIZATION BOUNDARY

A revision may transition through:

DRAFT
↓
VALIDATING
↓
VALIDATED
↓
DOCUMENTED
↓
READY_FOR_REVIEW
↓
ACCEPTED

The exact workflow remains configurable.

131. PROJECT STATE MACHINE

Project lifecycle may initially include:

DRAFT
ACTIVE
IN_REVIEW
ACCEPTED
ARCHIVED

Deletion is represented as a controlled lifecycle state where required.

132. COMMAND STATE MACHINE

Commands may transition:

PROPOSED
↓
VALIDATING
↓
APPROVED
↓
EXECUTING
↓
COMPLETED

Failure transitions to:

FAILED

Cancellation transitions to:

CANCELLED

133. JOB STATE MACHINE

Jobs use:

QUEUED



RUNNING



VALIDATING



COMPLETED

Failure:

FAILED

Cancellation:

CANCELLED

Stale work:

STALE

134. DEPLOYMENT ENVIRONMENTS

The system shall support:

development

test

staging

production

Each environment has independent:

database

object storage

queue

credentials

configuration

135. DEVELOPMENT DATA

Development and test data must never be confused with production data.

Seed records must identify themselves as:

DEVELOPMENT

or:

TEST

where appropriate.

136. BACKUP STRATEGY

PostgreSQL backups protect:

- project state
- revisions
- requirements
- relationships
- audit
- validation metadata
- acceptance records

Object-storage lifecycle and replication protect:

- drawings
- documents
- geometry artifacts
- source artifacts

The backup strategy must preserve the relationship between database records and stored objects.

137. DISASTER RECOVERY PRINCIPLE

Recovery must restore:

- database
- +
- object references
- +
- generated artifacts

to a mutually consistent state.

A database-only recovery is insufficient if document provenance is lost.

138. DATA INTEGRITY

Critical relationships require database foreign keys where practical.

Critical invariants require domain validation.

Critical historical records require immutability controls.

OneDraft therefore uses three integrity layers:

DATABASE INTEGRITY

+

DOMAIN INTEGRITY

+

AUDIT INTEGRITY

139. PERFORMANCE PRINCIPLE

Premature optimization is prohibited.

Initial performance priorities are:

correctness

determinism

traceability

transaction integrity

reproducibility

Performance optimization follows measured workloads.

140. CACHING PRINCIPLE

Caching is permitted for:

geometry

read-heavy project views

derived documentation

code-rule lookups

AI context

Caching must never supersede authoritative state.

141. ASYNCHRONOUS ARCHITECTURE

The system distinguishes:

synchronous operations

from:

asynchronous operations

Synchronous:

create project
read project
read model
submit command

Asynchronous:

geometry generation
large validation
drawing generation
document generation
code ingestion
large exports

142. EVENT-DRIVEN DERIVATION

A successful model change may emit:

ElementModified

which can trigger:

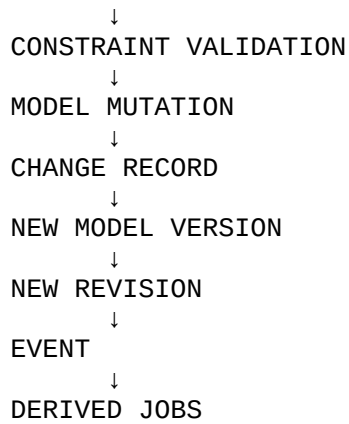
geometry regeneration
affected view regeneration
validation
drawing regeneration

The event pipeline must preserve revision identity.

143. MODEL CHANGE PIPELINE

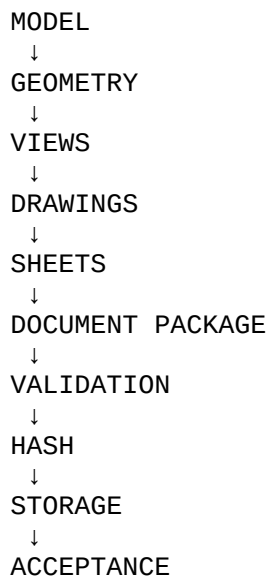
The complete mutation pipeline is:

USER / ARIA
↓
COMMAND
↓
AUTHORIZATION
↓
CURRENT REVISION CHECK
↓
DOMAIN VALIDATION



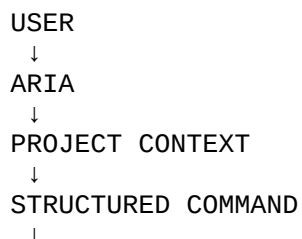
144. DOCUMENT PIPELINE

The complete documentation pipeline is:



145. ARIA + MODEL PIPELINE

The complete AI-assisted workflow is:



PROJECT MODEL
↓
CONSTRAINTS
↓
GEOMETRY
↓
VALIDATION
↓
DOCUMENTATION
↓
USER REVIEW

Aria is therefore integrated into the computational workflow rather than placed beside it as an independent chatbot.

146. EXTERNAL INTEGRATION BOUNDARY

Future integrations may include:

professional CAD applications
BIM applications
municipal submission systems
document management systems
survey systems
GIS
accounting
billing
identity providers

All integrations communicate through controlled interfaces.

147. EXPORT ARCHITECTURE

The export subsystem shall eventually support:

PDF
DXF
SVG
IFC
CSV
JSON

The initial implementation focuses on:

PDF

and establishes the export abstraction for future formats.

148. IMPORT ARCHITECTURE

Future import capabilities may include:

DXF
DWG
IFC
PDF
CSV
survey data
GIS data

Imported information must pass through normalization before becoming authoritative project-model data.

149. IMPORT SAFETY

Imported geometry shall not automatically become trusted project state.

The pipeline is:

```
IMPORT
↓
PARSE
↓
NORMALIZE
↓
VALIDATE
↓
MAP
↓
REVIEW
↓
PROJECT MODEL
```

150. SYSTEM OF RECORD

The OneDraft computational model remains the authoritative system of record.

The hierarchy is:

```
PROJECT MODEL
↓
REVISION HISTORY
↓
CODE MANIFEST
↓
```

VALIDATION



DOCUMENTATION



ACCEPTANCE



AUDIT

Derived artifacts can be regenerated.

Historical state remains preserved.

151. IMPLEMENTATION PRIORITY

Development shall proceed in dependency order:

1. Repository foundation
2. PostgreSQL migration
3. Domain types
4. Database repositories
5. Unit of Work
6. Project Model Service
7. API foundation
8. Project endpoints
9. Model endpoints
10. Command execution
11. Event/outbox system
12. Worker system
13. Geometry engine
14. Validation engine
15. Drawing engine
16. Document engine
17. Aria orchestration
18. Redline workflow
19. Acceptance workflow
20. End-to-end regression suite

152. FIRST VERTICAL SLICE

The first complete vertical slice shall be:

```
CREATE PROJECT
↓
CREATE BUILDING
↓
CREATE LEVEL
↓
CREATE SPACE
↓
CREATE WALL
↓
GENERATE GEOMETRY
↓
GENERATE PLAN
↓
GENERATE ARCH D SHEET
```

This proves the foundational OneDraft architecture.

153. SECOND VERTICAL SLICE

The second vertical slice shall be:

```
USER REQUEST
↓
ARIA
↓
MOVE WALL
↓
DOMAIN VALIDATION
↓
REVISION
↓
GEOMETRY REGENERATION
↓
PLAN REGENERATION
↓
ARCH D REGENERATION
```

This proves the AI-to-model-to-document loop.

154. THIRD VERTICAL SLICE

The third vertical slice shall be:

REDLINE
↓
ARIA
↓
COMMAND
↓
MODEL CHANGE
↓
VALIDATION
↓
DRAWING UPDATE
↓
REDLINE RESOLUTION

This proves iterative drafting.

155. FOURTH VERTICAL SLICE

The fourth vertical slice shall be:

PROJECT
↓
JURISDICTION
↓
CODE MANIFEST
↓
VALIDATION
↓
DRAWINGS
↓
DOCUMENT PACKAGE
↓
ACCEPTANCE

This proves the regulatory/document provenance chain.

156. ACCEPTANCE CRITERIA FOR v0.2

Version 0.2 is successful when the system can:

start all required services

connect to PostgreSQL

run migrations

create organizations

create users

create projects

create model objects
execute domain commands
create revisions
record events
submit jobs
process workers
generate geometry
generate a floor plan
compose an ARCH D sheet
accept a structured Aria command
apply a model modification
regenerate affected outputs
record a redline
validate a revision
generate a document package
hash the package
reconstruct historical state

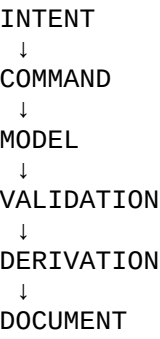
157. FAILURE CONDITIONS

The implementation is not considered complete if:

AI can bypass domain validation
AI can write directly to PostgreSQL
drawings become the system of record
historical revisions can be silently overwritten
stale worker output becomes current
code results cannot identify their source
documents cannot identify their originating revision
customer acceptance cannot identify the accepted package

158. ENGINEERING PRINCIPLE

The fundamental implementation rule is:



Never:



and never:



The drawing is an output.

The computational model is the product.

159. ONEDRAFT EXECUTION STACK

The complete stack now becomes:

USER / CLIENT
WEB APPLICATION
REST / API
AUTHORIZATION / SECURITY
APPLICATION SERVICES
ARIA LAYER
PROJECT MODEL DOMAIN

CONSTRAINT / RELATIONSHIP ENGINE
POSTGRESQL
EVENT / OUTBOX SYSTEM
JOB QUEUE
WORKERS
GEOMETRY ENGINE
COMPLIANCE ENGINE
VALIDATION ENGINE
DRAWING ENGINE
DOCUMENT ENGINE
OBJECT STORAGE
AUDIT / PROVENANCE

160. ARCHITECTURAL POSITION

OneDraft is now defined as an executable computational architecture in which:

Aria

provides intelligence,

Project Model

provides authoritative state,

PostgreSQL

provides persistence,

Geometry Engine

provides spatial computation,

Compliance Engine

provides rule evaluation,

Validation Engine

provides verification,

Drawing Engine

provides architectural representation,

Document Engine

provides deliverables,

and:

Revision / Audit

provides provenance.

161. NEXT MACHINE ARTIFACTS

The next executable package shall now be generated from this architecture.

The immediate files are:

```
001_initial_schema.sql
openapi.yaml
domain_types.py
commands.py
project_model_service.py
database.py
repositories.py
unit_of_work.py
main.py
projects.py
worker.py
docker-compose.yml
.env.example
```

These artifacts constitute the initial OneDraft runtime skeleton.

162. IMPLEMENTATION ORDER

The actual construction sequence is therefore:

```
REPOSITORY
  ↓
DATABASE
  ↓
DOMAIN
  ↓
API
  ↓
COMMAND SERVICE
  ↓
EVENTS
  ↓
WORKERS
```



Each layer must be executable before the next dependent layer is treated as operational.

163. VERSION 0.2 STATUS

ONEDRAFT EXECUTABLE APPLICATION STACK & IMPLEMENTATION ARCHITECTURE v0.2

STATUS: ESTABLISHED

The OneDraft persistence/API specification has now been translated into the executable application-stack architecture.

The system is no longer conceptually defined as merely:

DATABASE
+
API

It is defined as:

WEB
+
API
+
DOMAIN
+
ARIA
+
DATABASE
+
EVENTS
+
WORKERS
+
GEOMETRY
+
COMPLIANCE
+
VALIDATION

+
DRAWINGS
+
DOCUMENTS
+
AUDIT

The next implementation stage is therefore the construction of the actual repository artifacts, beginning with:

001_initial_schema.sql

followed by:

openapi.yaml
domain_types.py
commands.py
project_model_service.py

These become the first executable OneDraft software foundation.

164. FINAL IMPLEMENTATION PRINCIPLE

OneDraft shall be built from the inside outward.

The authoritative sequence is:

DOMAIN
↓
PERSISTENCE
↓
APPLICATION SERVICES
↓
API
↓
WORKERS
↓
DERIVED ENGINES
↓
ARIA
↓
USER INTERFACE

The user interface does not define the architecture.

The API does not define the project model.

Aria does not define the project model.

The drawing does not define the project.

The **Project Model** remains the central computational object from which the rest of OneDraft is derived.

That is the foundation upon which the executable OneDraft system will now be constructed.